【C言語入門講座】12章.構造体の基本的な使い方(初期化・代入・配列)

2023.09.12 2024.04.10

☜前の章へ　ポインタの基本的な使い方(宣言・代入・関数)
☞次の章へ　列挙型とは

【C言語入門講座】は全13章から構成される、初心者向けのC言語学習コンテンツです。1章から13章まで学習して頂くことでC言語の基礎を学習することができます。

この記事で分かること

1. 構造体を定義する方法
2. 構造体を宣言する方法
3. 構造体を初期化する3つの方法
4. 構造体のメンバ変数に値を代入する方法
5. 構造体のメンバ変数の値を出力する方法
6. 構造体のtypedefの使い方
7. 構造体の配列を宣言する方法
8. 構造体の配列へ値を代入する方法
9. 構造体の配列を初期化する方法

ここでは、基本的な構造体の使い方や構造体の配列の使い方を学ぶことが出来ます。

構造体とは

構造体とは、複数の異なるデータ型をひとつにまとめたデータ型のことです。

構造体は、関連するデータを1つの単位として管理するために使用され、複数の変数をまとめて1つのデータ構造として扱うことができます。

構造体の基本的な使い方

構造体の基本的な使い方

1. 構造体を定義する方法
2. 構造体を宣言する方法
3. 構造体を初期化する3つの方法
4. 構造体のメンバ変数に値を代入する方法
5. 構造体のメンバ変数の値を出力する方法

※上記の青い文字から読みたい項目へ移動することができます

①構造体を定義する方法

構造体の定義

```
struct 構造体名 {
型名 メンバ変数名;
};
```

C

```c
#include <stdio.h>

int main(void) {
    //構造体の定義
    struct Student {
        int num;
        char name[100];
    };

    return 0;
}
```

構造体を使用するためには、まず構造体を定義する必要があります。

構造体の定義をするためには、structと書いた後に構造体名を書きます。

今回は「struct Student」としています。

inr numやchar nameのことをメンバ変数と言います。

②構造体を宣言する方法

構造体の宣言

- struct 構造体名 変数名;

C

```c
#include <stdio.h>

int main(void) {
    //構造体の定義
    struct Student {
        int num;
        char name[100];
    };

    struct Student a;

    return 0;
}
```

構造体の宣言は、structと書いた後に構造体名を書き、変数名を書きます。

③構造体を初期化する3つの方法

構造体を0で初期化する方法

構造体を0で初期化する方法

- struct 構造体名 変数名 = {0};

C

```c
#include <stdio.h>

int main(void) {
    //構造体の定義
    struct Student {
        int num;
        char name[100];
    };

    struct Student a = {0};

    printf("番号:%d 名前:%s", a.num, a.name);


```

```
    return 0;
}
```

出力結果

```
番号:0 名前:
```

構造体を0で初期化したい場合は、{0}を代入します。

こうすることで全てのメンバに0が代入されます。

構造体を一括で初期化する方法

構造体を一括で初期化する方法

- struct 構造体名 変数名 = {値, 値, …};

C

```c
#include <stdio.h>

int main(void) {
    //構造体の定義
    struct Student {
        int num;
        char name[100];
    };

    struct Student a = {1, "Taro"};

    printf("番号:%d 名前:%s", a.num, a.name);

    return 0;
}
```

出力結果

```
番号:1 名前:Taro
```

構造体で定義した順に初期化したい値を書きます。

今回はnum、nameの順に記述します。

構造体の一部のメンバだけを初期化する方法

構造体の一部のメンバだけを初期化する方法

- struct 構造体名 変数名 = {.メンバ変数名 = 値, .メンバ変数名 = 値, …};

C

```c
#include <stdio.h>

int main(void) {
    //構造体の定義
    struct Student {
        int num;
        char name[100];
    };

    struct Student a = {.num = 1};

    printf("番号:%d 名前:%s", a.num, a.name);

    return 0;
}
```

出力結果

```
番号:1 名前:
```

特定のメンバ変数だけに初期化時に指定した値を代入したい場合、.(ドット)の後にメンバ変数名を書き、代入式を書きます。

代入していないメンバ変数は0で初期化されます。

④構造体のメンバ変数に値を代入する方法

構造体に代入する方法

- 変数名.メンバ変数名 = 値;

C

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    //構造体の定義
    struct Student {
        int num;
        char name[100];
    };

    struct Student a;

    //ピリオドを使い、変数のメンバにアクセスする
    a.num = 1;
```

```
    strcpy(a.name, "Taro");

    printf("番号:%d 名前:%s", a.num, a.name);

    return 0;
}
```

出力結果

```
番号:1 名前:Taro
```

変数名の後に.(ドット)を書き、メンバ変数名を書くことでメンバ変数にアクセスすることができます。

アクセスした状態で値を代入できます。

⑤構造体のメンバ変数の値を出力する方法

構造体のメンバ変数の値を出力する方法

- printf(“変換指定子”, 変数名.メンバ変数名);

C

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    //構造体の定義
    struct Student {
        int num;
        char name[100];
    };

    struct Student a;

    //ピリオドを使い、変数のメンバにアクセスする
    a.num = 1;
    strcpy(a.name, "Taro");

    printf("番号:%d 名前:%s", a.num, a.name);

    return 0;
}
```

出力結果

```
番号:1 名前:Taro
```

構造体のメンバ変数の値を出力したい場合、printf関数を使います。

左に変換指定子を書き、右に変数名.メンバ変数名を書きます。

構造体のtypedefの使い方

typedefの使い方

- typedef struct タグ名(省略可) {
  型名 メンバ変数名;
  } 構造体名;

C

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    //typedefを使った構造体の定義
    typedef struct {
        int num;
        char name[100];
    } Student;

    Student a;

    //ピリオドを使い、変数のメンバにアクセスする
    a.num = 1;
    strcpy(a.name, "Taro");

    printf("番号:%d 名前:%s", a.num, a.name);

    return 0;
}
```

出力結果

```
番号:1 名前:Taro
```

typedefを使うことで、構造体の名前である「Student」のみで構造体の型を表すことができます。

構造体の配列の基本的な使い方

構造体の配列の基本的な使い方

1. 構造体の配列を宣言する方法
2. 構造体の配列へ値を代入する方法
3. 構造体の配列を初期化する方法

※上記の青い文字から読みたい項目へ移動することができます

①構造体の配列を宣言する方法

構造体の配列の宣言

- struct 構造体名 変数名[要素数];

C

```c
#include <stdio.h>
#include <string.h>

// 構造体の定義
struct Student {
    int id;
    char name[100];
};

int main() {
    // 構造体の配列を宣言
    struct Student students[3]; // 3人分の学生データを保持する配列を宣言

    return 0;
}
```

構造体を宣言し、[要素数]を加えることで配列にすることができます。

②構造体の配列へ値を代入する方法

構造体の配列への代入

- 変数名[インデックス番号].メンバ変数名 = 値;

C

```c
#include <stdio.h>
#include <string.h>

// 構造体の定義
struct Student {
    int id;
    char name[100];
};

int main() {
    // 構造体の配列を宣言
    struct Student students[3]; // 3人分の学生データを保持する配列を宣言

    // 配列の各要素にデータを代入
    students[0].id = 1;
    strcpy(students[0].name, "一郎");

```

```
    students[1].id = 2;
    strcpy(students[1].name, "次郎");

    students[2].id = 3;
    strcpy(students[2].name, "三郎");

    // 配列の内容を表示
    for (int i = 0; i < 3; i++) {
        printf("学生%d: ID=%d, 名前=%s\n", i+1, students[i].id, students[i].name);
    }

    return 0;
}
```

出力結果

```
学生1: ID=1, 名前=一郎
学生2: ID=2, 名前=次郎
学生3: ID=3, 名前=三郎
```

インデックス番号の後に、.(ドット)とメンバ変数名を書くことで、代入することができます。

③構造体の配列を初期化する方法

構造体の配列の初期化

- struct 構造体名 変数名[] = {
  {値, 値, …},
  {値, 値, …},
  {値, 値, …}
  };

C

```
#include <stdio.h>

// 構造体の定義
struct Student {
    int id;
    char name[100];
};

int main() {
    // 構造体の配列を初期化
    struct Student students[] = {
        {1, "一郎"},
        {2, "次郎"},
        {3, "三郎"}
    };

    // 配列の内容を表示

```

```
    for (int i = 0; i < 3; i++) {
        printf("学生%d: ID=%d, 名前=%s\n", i+1, students[i].id, students[i].name);
    }

    return 0;
}
```

出力結果

```
学生1: ID=1, 名前=一郎
学生2: ID=2, 名前=次郎
学生3: ID=3, 名前=三郎
```

構造体の配列の初期化は通常の配列のように行うことが出来ます。

練習問題

①

以下の表をもとに構造体を宣言しましょう。
誰の成績を参照しますか？とコンソールから入力を促します。
指定された人の成績を表示します。

出席番号	名前	国語	算数	社会	理科
1	太郎	50	64	98	63
2	花子	55	59	75	74
3	一郎	80	77	83	88
4	桃子	71	67	77	73
5	二郎	45	90	44	92

＋ 解答例

☜前の章へ　ポインタの基本的な使い方(宣言・代入・関数)
☞次の章へ　列挙型とは